



### INTELLIGENT BANK VAULT SECURITY SYSTEM

| STUDENT ID | NAME                                        | GROUP     |
|------------|---------------------------------------------|-----------|
| 2024275932 | SOFEA FARISHA BINTI CHE ZULMAIDI            | CDCS2554B |
| 2024276562 | ALYA SOPHEA BINTI SHAHRUL MUNIR             | CDCS2554B |
| 2024276376 | NUR AIN NASUHA BINTI ABDUL FATAH            | CDCS2554B |
| 2024284686 | MUHAMMAD ADAM DANIAL AFIZAL BIN ABDUL HAFIZ | CDCS2554B |

## TABLE OF CONTENT

|                                             |    |
|---------------------------------------------|----|
| 1.0 EXECUTIVE SUMMARY.....                  | 3  |
| 2.0 SYSTEM REQUIREMENT.....                 | 4  |
| 2.1 Hardware Requirements.....              | 4  |
| 2.2 Software Requirement.....               | 5  |
| 3.0 SYSTEM DESIGN .....                     | 5  |
| 3.1 IOT Controlled System Flow Process..... | 5  |
| 3.2 Block Diagram.....                      | 5  |
| 3.3 Schematic Circuit Design.....           | 5  |
| 3.4 Flow Chart.....                         | 6  |
| 4.0 DEVELOPMENT PROCESS.....                | 6  |
| 4.1 Steps/Procedure.....                    | 6  |
| 4.2 Coding.....                             | 7  |
| 4.2.1 Role-Based Schedule Check.....        | 7  |
| 4.2.2 Gas Sensor Analog Threshold.....      | 7  |
| 4.2.3 Lockout Mechanism.....                | 7  |
| 4.2.4 Flask HTTP POST Logging.....          | 7  |
| 4.2.5 NTP Real-Time Clock.....              | 8  |
| 4.2.6 Sensor Cooldown Timers.....           | 8  |
| 5.0 TESTING AND RESULT.....                 | 8  |
| 5.1 Prototype .....                         | 8  |
| 5.2 Testing .....                           | 9  |
| 5.3 Result.....                             | 9  |
| 5.2.1 Dashboard Screenshots.....            | 9  |
| 5.2.2 Telegram Alert Screenshots.....       | 9  |
| 5.2.3 AI Risk Analysis .....                | 9  |
| 5.3 Costing .....                           | 10 |
| 5.4 Lean Business Model Canvas .....        | 10 |
| 6.0 CONCLUSION.....                         | 10 |
| 6.1 Recommendation .....                    | 11 |
| 6.2 Problem Solution .....                  | 11 |
| 7.0 REFERENCES .....                        | 11 |

### **Table of Figure**

|                                                                 |    |
|-----------------------------------------------------------------|----|
| Figure 1: System Flow Diagram                                   | 6  |
| Figure 2: Block Diagram                                         | 7  |
| Figure 3: Circuit Diagram                                       | 8  |
| Figure 4: Flow Chart for Intelligent Bank Vault Security System | 9  |
| Figure 5: Intelligent Bank Vault Prototype                      | 14 |
| Figure 6: Live Dashboard screenshots                            | 16 |
| Figure 7: Telegram Alert screenshots                            | 17 |
| Figure 8: AI Risk Analysis screenshots                          | 18 |

## 1.0 EXECUTIVE SUMMARY

Traditionally, bank vault security systems have relied on manual logbooks, physical keys and human guards, each of which are slow, inflexible and unable to react quickly to threats. A more intelligent, automated security solution is becoming essential as financial institutions manage high-value assets.

In order to provide automated access control, real-time threat detection and structured activity logging, this project presents an **Intelligent Bank Vault Security System** that combines IoT sensors, rule-based AI and a centralized database. All of these functions are handled by a single ESP32 microcontroller with integrated Wi-Fi.

The system uses a 4-digit PIN authentication procedure and keypad-based user selection. Role-based scheduling controls access: Cleaners are only allowed admittance in the evenings (6 PM–10 PM), but managers can enter the vault at any time. Staff are allowed entry on weekdays from 9 AM to 6 PM. Any access that occurs outside of these allowed times is considered as a MEDIUM risk and an unusual hour event.

Using rule-based logic that compares credentials, access time, motion detection and gas sensor readings, an AI behavior analysis module continuously analyzes system activity and assigns a risk prediction level LOW, MEDIUM or HIGH based on each access occurrence.

User name, role, access time, motion detection, gas detection, warning kind and level of risk are all recorded in a SQLite database on the laptop. Assessors and security personnel can access a live-updating log table via a lightweight Flask online dashboard hosted at a local IP address, doing away with the necessity for manual record keeping.

### 1.1 Objectives

To address the issues of traditional vault security systems, this project aims to achieve the following specific objectives :

- a) To use 4-digit PIN entry and keypad user selection for automated two-factor authentication.
- b) To implement role-based access scheduling: Cleaner (evenings 6 PM–10 PM), Manager (anytime) and Staff (weekdays 9 AM–6 PM).
- c) To automatically lock out after three continuous incorrect PIN tries in order to identify and react to intrusion attempts.
- d) To use a MQ-2 sensor with analog threshold detection to keep an eye on environmental risks (such as smoke and gas) within the vault.
- e) To utilize a Telegram bot for sending security staff HIGH and MEDIUM risk notifications in real time.
- f) To automatically record every access event in a SQLite database and show it on a live Flask web dashboard.
- g) To classify intelligent threats using rule-based AI risk rating (LOW, MEDIUM and HIGH).

## 2.0 SYSTEM REQUIREMENT

### 2.1 Hardware Requirements

| Component       | Model                        | Quantity | Role in System                                                                                                                               |
|-----------------|------------------------------|----------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Microcontroller | ESP32 Dev Module             | 1        | Main processor, it handles all logic, Wi-Fi, sensors and outputs                                                                             |
| Keypad          | 4x3 Matrix                   | 1        | User selection (1/2/3) and 4-digit PIN entry                                                                                                 |
| LCD display     | 16x2 I2C (0x27)              | 1        | Shows system status, risk level and messages                                                                                                 |
| Door actuator   | SG90 Micro Servo             | 1        | Physically opens and closes vault door on valid access                                                                                       |
| Motion sensor   | PIR HC-SR501                 | 1        | Detects human movement without authentication                                                                                                |
| Gas sensor      | MQ-2 (AO pin)                | 1        | Detects gas or smoke above analog threshold value 600                                                                                        |
| Alarm           | Active Buzzer Module         | 1        | Audio feedback: <ul style="list-style-type: none"><li>- 1 beep = granted</li><li>- 3 beeps = denied</li><li>- continuous = lockout</li></ul> |
| Jumper wires    | Male-to-Male, Male-to-Female | 30+      | Component interconnection on breadboard                                                                                                      |
| Breadboard      | Full-size 830 tie-points     | 1        | Component mounting and power distribution                                                                                                    |
| Power supply    | USB cable to laptop          | 1        | Powers ESP32 and all connected components                                                                                                    |
| Enclosure       | PVC board                    | 1        | Physical vault prototype housing                                                                                                             |

## 2.2 Software Requirement

| Software/Platform    | Purpose                                               |
|----------------------|-------------------------------------------------------|
| Arduino IDE          | ESP32 sketch development, compilation and upload      |
| ESP32 board package  | ESP32 board support for Arduino IDE                   |
| UniversalTelegramBot | Telegram bot message sending via API                  |
| ArduinoJson          | JSON parsing for Telegram API responses               |
| Keypad library       | 4x3 matrix keypad scanning                            |
| LiquidCrystal_I2C    | 16x2 LCD I2c display control                          |
| ESP32Servo           | SG90 servo motor PWM control on ESP32                 |
| Python 3.x           | Flask server and database scripts                     |
| Flask                | Web server for live dashboard and /log POST endpoint  |
| SQLite3              | Lightweight local database for access logs            |
| Telegram app         | Receiving real-time security alerts from bot          |
| Kaggle               | Dataset hosting and AI risk pattern analysis notebook |
| WPS Office/Excel     | Viewing exported vault_logs.csv dataset               |

### 3.0 SYSTEM DESIGN

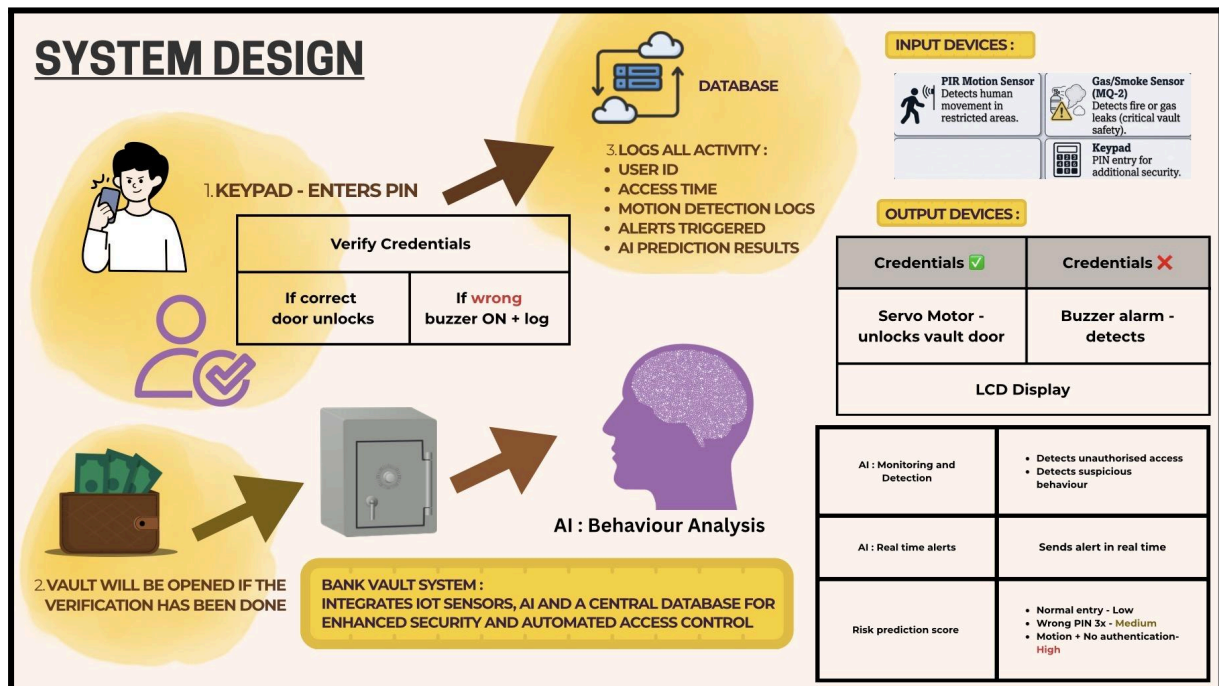


Figure 1: System Design

#### 3.1 IOT Controlled System Flow Process

Through the ESP32 microcontroller, all sensors, actuators, communication modules, and database systems collaborate to operate the Intelligent Bank Vault Security System as a continuous Internet of Things loop. Three levels can be used to characterize the system flow:

##### Layer 1 : Authentication

- The user presses key 1, 2 or 3 on the keypad to select their profile (Staff / Manager / Cleaner).
- The user enters their 4-digit PIN. ESP32 verifies against stored credentials.
- If the wrong PIN entered 3 times, automatic 30-second lockout is triggered.

### Layer 2 : AI Risk Scoring

- ESP32 retrieves current time from NTP server (GMT+8 Malaysia).
- Role-based scheduling check: Does the user's authorized window contain the current hour?
- Is motion detected by the PIR sensor without authentication?
- Does the analog value on the MQ-2 AO pin exceed the 600 threshold?
- Risk score assigned: LOW (normal access), MEDIUM (unusual hour), HIGH (lockout/motion/gas).

### Layer 3 : Response and Logging

- Output actions triggered based on risk level: servo opens/stays closed, buzzer pattern, LCD message.
- Telegram push notification sent to security personnel for MEDIUM and HIGH risk events.
- HTTP POST request sent from ESP32 to Flask server on laptop event written to SQLite database.
- Flask dashboard auto-refreshes every 5 seconds to show an updated live log table.

## 3.2 Block Diagram

Example:

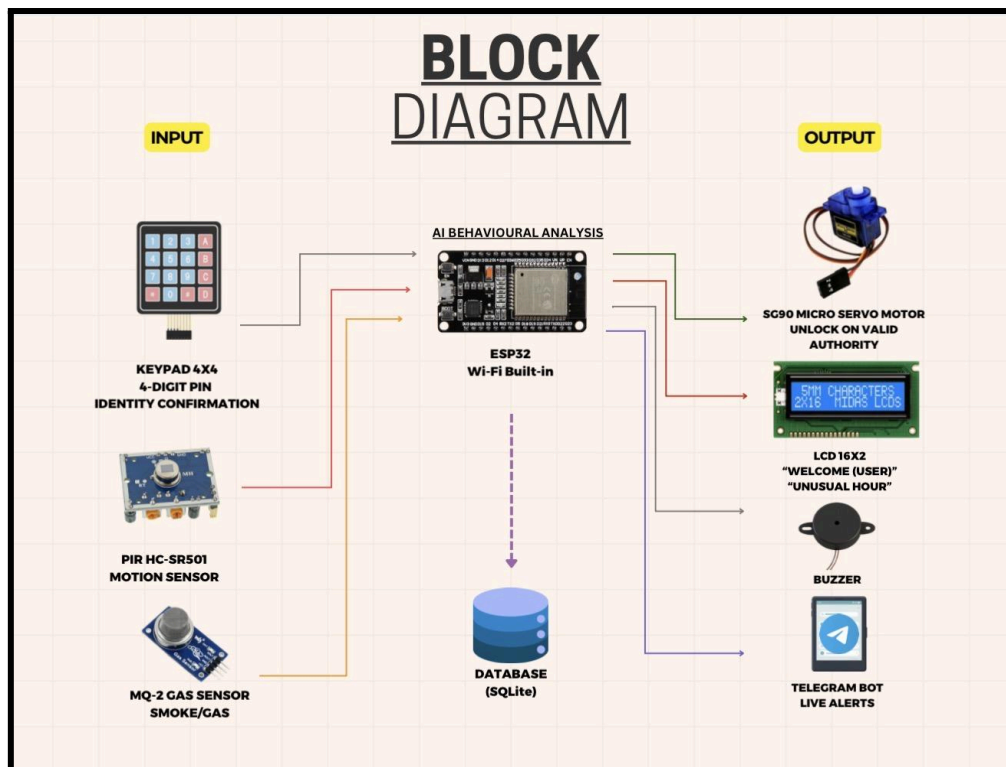


Figure 2: Block Diagram



1. *Input Connections (Sensors to ESP32)*
  - **4x3 Keypad** : Connected through multiple GPIO lines directly into the **ESP32** to transmit keypress data.
  - **PIR motion Sensor (HC-SR501)** : Connected through a single digital GPIO input pin on the **ESP32** to send HIGH/LOW motion signal states.
  - **MQ-2 Gas Sensor** : Connected to an analog input pin on the **ESP32** to transmit gas or smoke concentration levels.
2. *Central Processing and Database Connection*
  - **ESP32 to SQLite Database**: Linked through a bidirectional software data pipeline (indicated by the dashed arrow), allowing the **ESP32** to write activity logs and read user credential data.
3. *Output Connections (ESP32 to Peripherals and Cloud)*
  - **SG90 Servo Motor** : Connected to a GPIO pin on the ESP32 to control physical angle position.
  - **16x2 LCD Display**: Connected to the **ESP32** to receive display text parameters.
  - **Buzzer**: Connected to a digital GPIO output pin on the **ESP32** to trigger audio frequencies.
  - **Telegram Bot**: Connected wirelessly from the **ESP32** over its built-in Wi-Fi interface out to the internet to trigger cloud alert messages.

### 3.3 Schematic Circuit Design

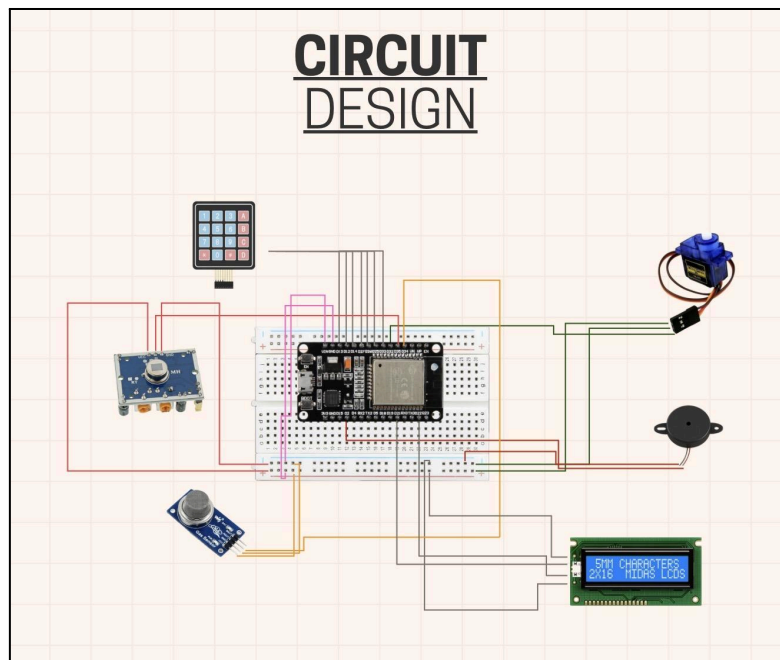


Figure 3: Circuit Diagram

### 1. Power distribution :

- **5V / VIN Rail:** Supplies power to the MQ-2 Gas Sensor, LCD I2C interface, PIR Sensor (3.3V or 5V), and Servo Motor (external supply recommended).
- **Common Ground (GND):** Shared across all peripherals, sensors, and the ESP32 board to establish a unified reference voltage.

### 2. Sensor Inputs to ESP32 :

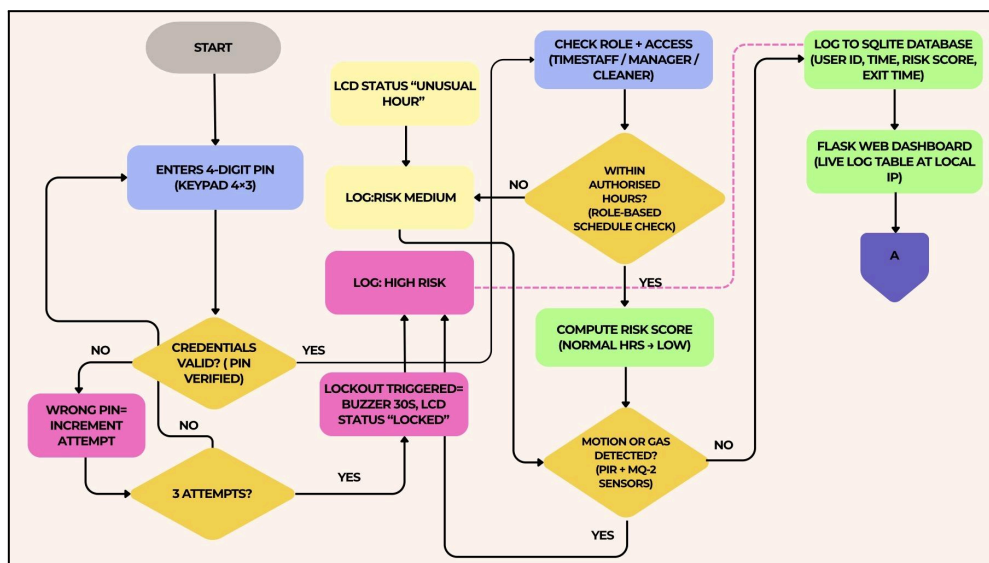
- **Keypad (4x3 matrix):** Connected across **7 GPIO pins** (Rows 1–4 to GPIO 13, 12, 14, 27; Columns 1–3 to GPIO 26, 25, 33).
- **PIR Motion Sensor:** Output signal (**OUT**) connected to input-only **GPIO 35**.
- **MQ-2 Gas Sensor:** Analog output (**A0**) connected to pin **GPIO 34**.

### 3. Output Actuators & Displays :

- **LCD Display (I2C):** Interfaces via pins **SDA (GPIO 21)** and **SCL (GPIO 22)**.
- **Servo Motor:** PWM control signal pin connected to **GPIO 32**.
- **Buzzer:** Signal pin connected to digital output **GPIO 2**.

## 3.4 Flow Chart

Example:



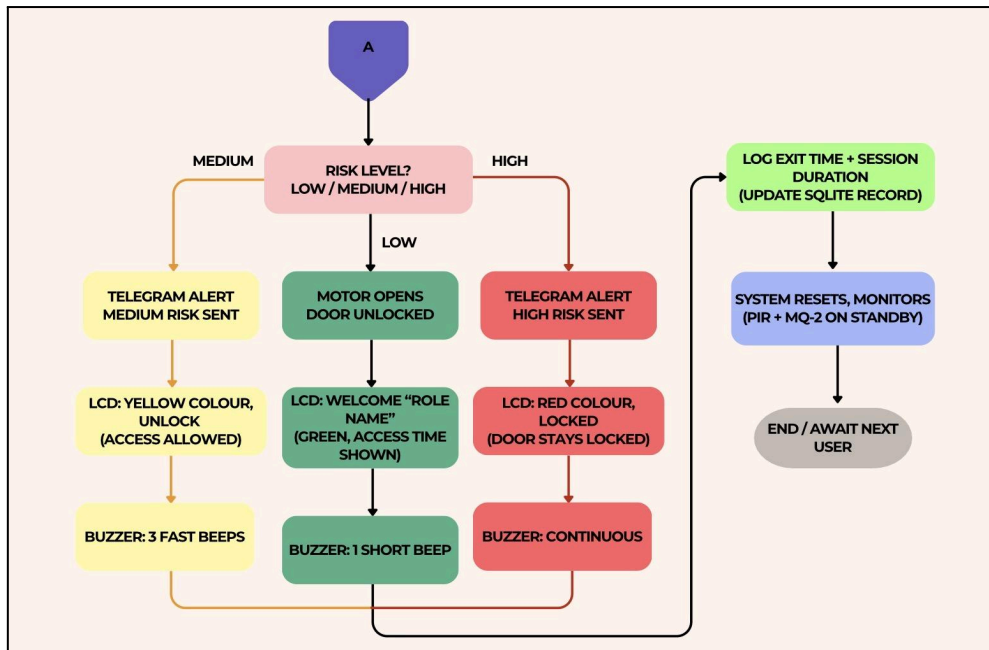


Figure 4: Flow Chart for Intelligent Bank Vault Security System

## 4.0 DEVELOPMENT PROCESS

### 4.1 Steps/Procedure

| Step | Activity                    | Description                                                                                                                                                                                                                                                                                  |
|------|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Project planning            | Defined system requirements, identified all hardware and software components, assigned team roles, and planned the PVC board prototype enclosure dimensions.                                                                                                                                 |
| 2    | Component testing           | Tested each component individually: LCD I2C address scan, keypad mapping, servo rotation, PIR sensitivity adjustment, MQ-2 analog calibration with threshold set at 600 and buzzer pattern testing.                                                                                          |
| 3    | Database setup              | Created vault.db using setup_db.py with users and logs tables. Populated with 3 test users. Generated 150 realistic log records for demonstration and Kaggle analysis.                                                                                                                       |
| 4    | ESP32 sketch development    | Built main sketch integrating keypad selection, PIN verification, role-based scheduling, AI risk scoring, buzzer patterns, servo control, LCD feedback, gas analog threshold detection, PIR motion detection, NTP time sync, Telegram alerts, HTTP POST to Flask and sensor cooldown timers. |
| 5    | Wi-Fi and NTP integration   | Connected ESP32 to WiFi hotspot. Synced real-time clock via NTP (GMT+8). Live time used for role-based hour checking and accurate timestamp generation for all log entries                                                                                                                   |
| 6    | Telegram bot setup          | Created bot via BotFather, obtained token and chat ID, integrated UniversalTelegramBot library. Configured alerts for MEDIUM risk and HIGH risk events, including user name, role, event type, and timestamp.                                                                                |
| 7    | Flask dashboard development | Built app.py with /log (POST endpoint) and / (dashboard). Built dashboard.html with live log table, colour-coded risk badges and summary cards for Total, LOW, MEDIUM, HIGH counts.                                                                                                          |
| 8    | System integration testing  | Tested all 5 scenarios end to end: normal LOW, MEDIUM unusual hour, HIGH                                                                                                                                                                                                                     |

|    |                               |                                                                                                                                                                                                                                                  |
|----|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    |                               | lockout, PIR motion without authentication, and gas analog alert above threshold 600. Verified Flask receives POST (response code 200), dashboard updates correctly, and Telegram alerts fire with correct content and timing                    |
| 9  | Dataset generation and Kaggle | Generated 150 realistic log records. Exported to vault_logs.csv. Uploaded to Kaggle for AI risk pattern analysis with 4 visualisation charts.                                                                                                    |
| 10 | Prototype assembly            | Assembled all components into a PVC board enclosure. Front door: keypad, LCD, servo. Inside vault: PIR, MQ-2, active buzzer, ESP32. Wires routed through the cable slot at bottom edge and bundled neatly with tape labels for cable management. |

## 4.2 Coding

The main sketch esp32\_main.ino is the core of the system. Key code sections are shown below.

### 4.2.1 Role-Based Schedule Check and AI Risk Scoring

```
// Check if current hour is within user's authorised window bool
authorised = (currentHour >= currentUser.allowedStart &&
currentHour < currentUser.allowedEnd); // AI Risk Scoring –
rule-based classification String risk = "LOW"; if (!authorised)
risk = "MEDIUM"; // outside hours if (motionDetected &&
!authorised) risk = "HIGH"; // motion + wrong time
Serial.println("User: " + currentUser.name + " | Role: " +
currentUser.role + " | Hour: " + String(currentHour) + " | Auth: "
+ String(authorised) + " | Risk: " + risk);
```

### 4.2.2 Gas Sensor Analog Threshold Detection

```
#define GAS_THRESHOLD 600 // Calibrated threshold for MQ-2 AO pin
// Read analog value from MQ-2 AO pin (range: 0-4095 on ESP32) int
gasValue = analogRead(MQ2_AO_PIN); bool gasDetected = (gasValue >
GAS_THRESHOLD); // Clean air typically reads 200-400. Gas/smoke
reads 600+ Serial.println("Gas AO value: " + String(gasValue));
```

#### 4.2.3 Automatic Lockout Mechanism

```
// Trigger lockout after 3 consecutive wrong PIN attempts if
(wrongPinCount >= 3) { lockedOut = true; lockoutStart = millis();
userSelected = false; selectedUser = -1; lcd.clear();
lcd.print("!! LOCKED !!"); buzzPattern("lockout"); // continuous
buzzer sendTelegram(msg); // HIGH RISK Telegram alert
sendToFlask(..., "HIGH", "PIN_LOCKOUT", ...); } // Auto-reset
after LOCKOUT_DURATION (30 seconds) if (lockedOut && (millis() -
lockoutStart >= LOCKOUT_DURATION)) { lockedOut = false;
wrongPinCount = 0; showIdleScreen(); }
```

#### 4.2.4 Flask HTTP POST Logging

```
void sendToFlask(String userName, String role, String risk, bool
motion, bool gas, String alert, String timestamp) { HTTPClient
http; http.begin(flaskURL); // http://LAPTOP_IP:5000/log
http.addHeader("Content-Type", "application/json"); String body =
""; body += "\"user_name\": \"" + userName + "\", "; body +=
 "\"role\": \"" + role + "\", "; body += "\"risk_score\": \"" + risk +
 "\", "; body += "\"motion_detected\": " + String(motion ? "1":"0") +
 ", "; body += "\"gas_detected\": " + String(gas ? "1":"0") + ", ";
body += "\"alert_triggered\": \"" + alert + "\", "; body +=
 "\"access_time\": \"" + timestamp + "\"}"; int responseCode =
http.POST(body); // 200 = success | -1 = connection failed (check
IP/firewall) http.end(); }
```

#### 4.2.5 NTP Real-Time Clock and Sensor Cooldown

```
// Sync to Malaysian time GMT+8 via NTP on startup
configTime(28800, 0, "pool.ntp.org"); // Cooldown timers prevent
repeated Telegram spam unsigned long lastMotionAlert = 0; const
unsigned long MOTION_COOLDOWN = 10000; // 10 seconds unsigned long
lastGasAlert = 0; const unsigned long GAS_COOLDOWN = 15000; // 15
seconds // Only fire alert if cooldown period has elapsed if
(motionDetected && (now - lastMotionAlert > MOTION_COOLDOWN)) {
lastMotionAlert = now; // trigger HIGH risk alert }
```

## 5.0 TESTING AND RESULT

### 5.1 Prototype

Example:



Figure 5: Intelligent Bank Vault Prototype

### 5.2 Testing

To verify the whole functionality of the system, five essential test scenarios were carried out. Every scenario was manually initiated, and every output channel showed the relevant system response: Flask dashboard, servo motor, buzzer, LCD screen and Telegram bot.

| # | Scenario                   | Action Taken                                                                   | Expected Behaviour                                                                            |
|---|----------------------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| 1 | Normal LOW Risk Access     | Press key 1, enter PIN 1234 during authorised hours (9AM-6PM)                  | LCD: Access Granted   LOW. Servo opens 90 deg. 1 short beep. Dashboard: new LOW row.          |
| 2 | MEDIUM Risk – Unusual Hour | Press key 3 (Cleaner), enter PIN 9012 during daytime (outside 6PM-10PM window) | Telegram MEDIUM RISK alert sent first. LCD: Unusual Hour. Servo opens. Dashboard: MEDIUM row. |

|   |                         |                                                                    |                                                                                                                 |
|---|-------------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| 3 | HIGH Risk – PIN Lockout | Press key 1, enter wrong PIN 0000 three consecutive times          | LCD: LOCKED.<br>Continuous buzzer for 30s. Telegram HIGH RISK alert. Dashboard: HIGH row. Auto-reset after 30s. |
| 4 | HIGH Risk – PIR Motion  | Wave hand in front of PIR sensor while idle screen is showing      | LCD: Motion Detected. 3 fast beeps. Telegram HIGH RISK alert. Dashboard: HIGH row. 10s cooldown.                |
| 5 | HIGH Risk – Gas Alert   | Bring gas source near MQ-2 AO pin until analog reading exceeds 600 | LCD: GAS ALERT EVACUATE NOW.<br>Continuous buzzer. Telegram HIGH alert with gas level. Dashboard: HIGH row.     |

### 5.3 Result

All five test scenarios passed successfully. The table below summaries the complete test results:

| Test Scenario                 | Expected Result                           | Status |
|-------------------------------|-------------------------------------------|--------|
| Correct PIN within hours      | LOW risk, servo opens, 1 beep             | PASS   |
| Correct PIN outside hours     | MEDIUM risk, Telegram alert, door opens   | PASS   |
| Wrong PIN x 3                 | Lockout 30s, HIGH risk, continuous buzzer | PASS   |
| Motion without authentication | HIGH risk, 3 beeps, Telegram alert        | PASS   |
| Gas above threshold 600       | GAS ALERT, evacuate, Telegram HIGH        | PASS   |
| Flask dashboard update        | New row appears within 5 seconds          | PASS   |
| Telegram LOW risk             | No alert sent (correct behaviour)         | PASS   |
| Telegram MEDIUM risk          | Alert sent before door opens              | PASS   |
| Telegram HIGH risk            | Alert sent immediately                    | PASS   |



## 5.2.1 Dashboard Screenshots

| <div> <div>323<br/>Total Events</div> <div>77<br/>Low Risk</div> <div>64<br/>Medium Risk</div> <div>182<br/>High Risk</div> </div> |             |       |                     |           |          |        |     |                |      |
|------------------------------------------------------------------------------------------------------------------------------------|-------------|-------|---------------------|-----------|----------|--------|-----|----------------|------|
| #                                                                                                                                  | USER        | ROLE  | ACCESS TIME         | EXIT TIME | DURATION | MOTION | GAS | ALERT          | RISK |
| 323                                                                                                                                | Unknown     | N/A   | 2026-07-24 11:03:36 | —         | —        | Yes    | No  | MOTION_NO_AUTH | HIGH |
| 322                                                                                                                                | Unknown     | N/A   | 2026-07-24 11:03:16 | —         | —        | Yes    | No  | MOTION_NO_AUTH | HIGH |
| 321                                                                                                                                | Unknown     | N/A   | 2026-07-22 13:37:12 | —         | —        | Yes    | No  | MOTION_NO_AUTH | HIGH |
| 320                                                                                                                                | Unknown     | N/A   | 2026-07-22 13:37:00 | —         | —        | Yes    | No  | MOTION_NO_AUTH | HIGH |
| 319                                                                                                                                | Unknown     | N/A   | 2026-07-22 13:36:47 | —         | —        | Yes    | No  | MOTION_NO_AUTH | HIGH |
| 318                                                                                                                                | Unknown     | N/A   | 2026-07-22 13:36:35 | —         | —        | Yes    | No  | MOTION_NO_AUTH | HIGH |
| 317                                                                                                                                | Unknown     | N/A   | 2026-07-22 13:36:18 | —         | —        | Yes    | No  | MOTION_NO_AUTH | HIGH |
| 316                                                                                                                                | Ahmad Staff | Staff | 2026-07-22 13:36:04 | —         | —        | No     | No  | PIN_LOCKOUT    | HIGH |
| 315                                                                                                                                | Unknown     | N/A   | 2026-07-22 13:35:44 | —         | —        | Yes    | No  | MOTION_NO_AUTH | HIGH |
| 314                                                                                                                                | Unknown     | N/A   | 2026-07-22 13:35:32 | —         | —        | Yes    | No  | MOTION_NO_AUTH | HIGH |
| 313                                                                                                                                | Unknown     | N/A   | 2026-07-22 13:35:18 | —         | —        | Yes    | No  | MOTION_NO_AUTH | HIGH |

| #   | USER        | ROLE  | ACCESS TIME         | EXIT TIME | DURATION | MOTION | GAS | ALERT          | RISK |
|-----|-------------|-------|---------------------|-----------|----------|--------|-----|----------------|------|
| 324 | Unknown     | N/A   | 2026-07-24 11:04:35 | —         | —        | Yes    | No  | MOTION_NO_AUTH | HIGH |
| 323 | Unknown     | N/A   | 2026-07-24 11:03:36 | —         | —        | Yes    | No  | MOTION_NO_AUTH | HIGH |
| 322 | Unknown     | N/A   | 2026-07-24 11:03:16 | —         | —        | Yes    | No  | MOTION_NO_AUTH | HIGH |
| 321 | Unknown     | N/A   | 2026-07-22 13:37:12 | —         | —        | Yes    | No  | MOTION_NO_AUTH | HIGH |
| 320 | Unknown     | N/A   | 2026-07-22 13:37:00 | —         | —        | Yes    | No  | MOTION_NO_AUTH | HIGH |
| 319 | Unknown     | N/A   | 2026-07-22 13:36:47 | —         | —        | Yes    | No  | MOTION_NO_AUTH | HIGH |
| 318 | Unknown     | N/A   | 2026-07-22 13:36:35 | —         | —        | Yes    | No  | MOTION_NO_AUTH | HIGH |
| 317 | Unknown     | N/A   | 2026-07-22 13:36:18 | —         | —        | Yes    | No  | MOTION_NO_AUTH | HIGH |
| 316 | Ahmad Staff | Staff | 2026-07-22 13:36:04 | —         | —        | No     | No  | PIN_LOCKOUT    | HIGH |
| 315 | Unknown     | N/A   | 2026-07-22 13:35:44 | —         | —        | Yes    | No  | MOTION_NO_AUTH | HIGH |
| 314 | Unknown     | N/A   | 2026-07-22 13:35:32 | —         | —        | Yes    | No  | MOTION_NO_AUTH | HIGH |
| 313 | Unknown     | N/A   | 2026-07-22 13:35:18 | —         | —        | Yes    | No  | MOTION_NO_AUTH | HIGH |
| 312 | Unknown     | N/A   | 2026-07-22 13:35:07 | —         | —        | Yes    | No  | MOTION_NO_AUTH | HIGH |

Figure 6: Live Dashboard Screenshots

### 5.2.2 Telegram Alert Screenshots

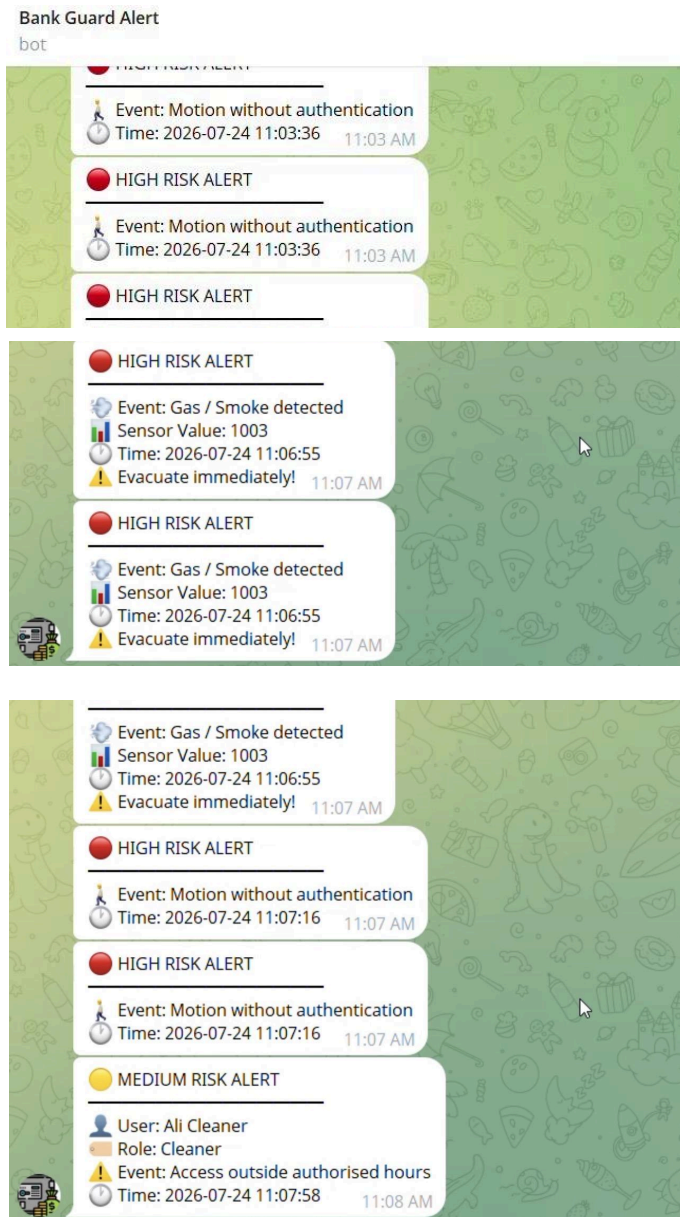


Figure 7: Telegram Alert Screenshots

5.2.3 AI Risk Analysis

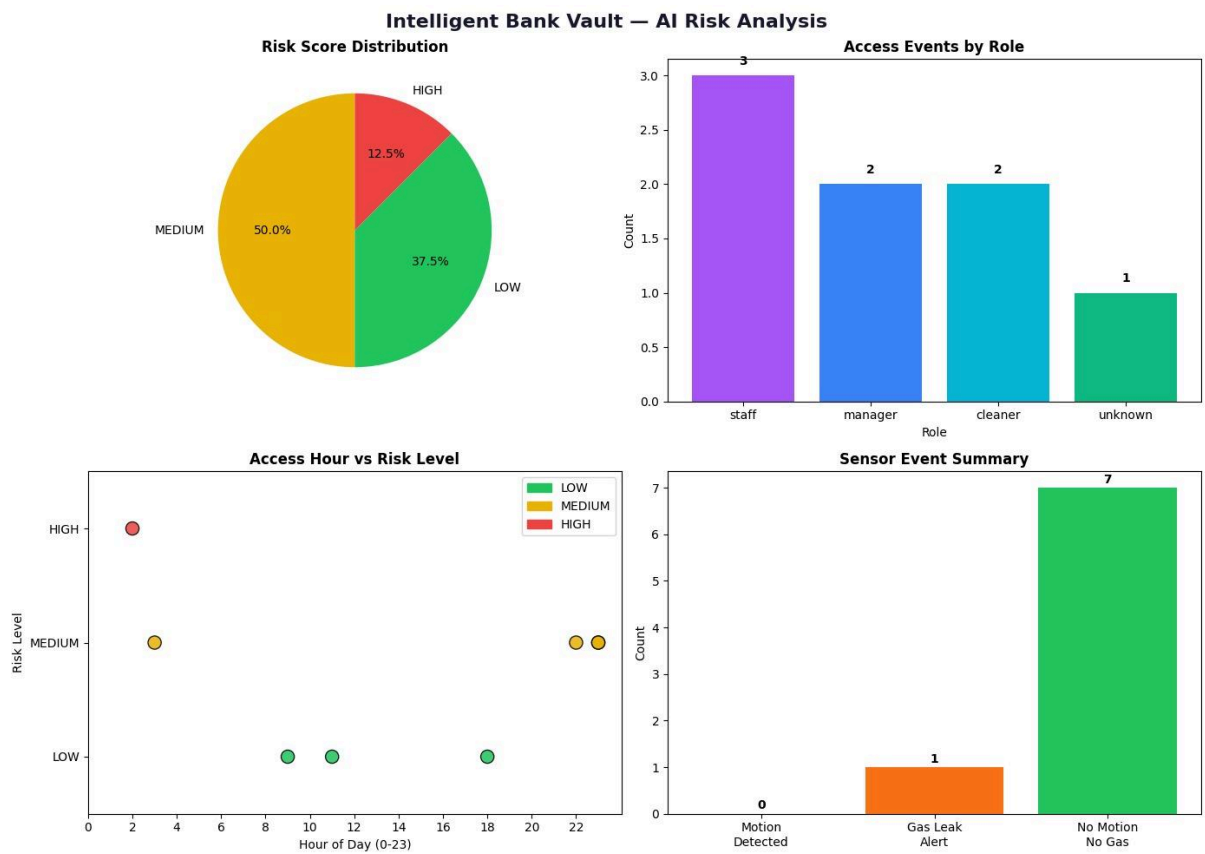
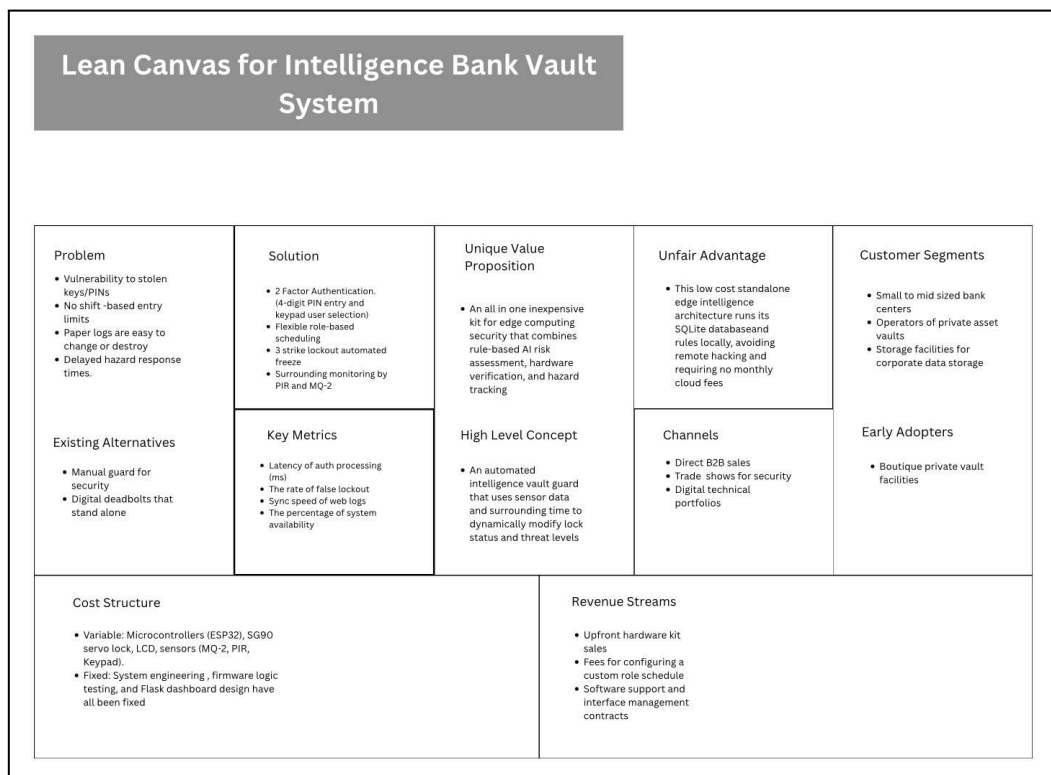


Figure 8: AI Risk Analysis Screenshots

## 5.3 Costing

| Component        | Model                           | Estimated Cost (RM) |
|------------------|---------------------------------|---------------------|
| ESP32 Dev Module | ESP32 WROOM-32                  | 25.00               |
| Keypad           | 4x3 Matrix                      | 5.00                |
| LCD Display      | 16x2 I2C                        | 8.00                |
| SG90 Servo Motor | SG90 Micro Servo                | 6.00                |
| PIR Sensor       | HC-SR501                        | 5.00                |
| Gas Sensor       | MQ-2                            | 8.00                |
| Active Buzzer    | 3-pin Module                    | 3.00                |
| Jumper Wires     | Male-to-Male,<br>Female-to-Male | 5.00                |
| Breadboard       | 830 tie-points                  | 8.00                |
| PVC Board        | 30x20x20 cm sheets              | 10.00               |
| Miscellaneous    | Hot glue, tape                  | 5.00                |

## Lean Business Model Canvas



## 6.0 CONCLUSION

The Intelligent Bank Vault Security System successfully shows how more intelligent, automated solutions can take the place of outdated security infrastructure using reasonably priced IoT technology. The system achieves complete vault protection without requiring continuous human surveillance by combining keypad authentication, role-based scheduling, PIR motion detection, MQ-2 gas monitoring, AI risk scoring, real-time Telegram alerts, SQLite logging and a live Flask dashboard through a single ESP32.

Based on credentials, time and sensor data, the rule-based AI engine accurately categorizes access events as LOW, MEDIUM or HIGH risk. The automated logging lowers the possibility of logbook forgery and does away with manual record-keeping. No matter where they are physically located, security professionals are guaranteed to receive rapid notifications thanks to the Telegram alert system.

Compared to conventional vault systems, this idea provides a more clever, safe and affordable security option. Through automation and real-time intelligence, it is a practical and scalable solution for modern banking environments, greatly decreasing the workload of security professionals while simultaneously enhancing protection for critical assets.

### 6.1 Recommendation

- **RFID Integration:** RFID card scanning can be used in place of keypad user selection for more secure and realistic user identification. Every card UID was associated with a database user profile.
- **Cloud Database:** To enable multi-branch deployment, remote log access and persistent data between sessions, switch from local SQLite to a cloud database (Firebase or MySQL).
- **Facial Recognition:** Add a camera module (OV2640) with ESP32-CAM for a third authentication factor, increasing security to three-factor authentication.
- **Mobile App Dashboard:** Provide a specialized mobile application that allows security staff to remotely lock or unlock the vault, examine real-time logs and receive warnings.
- **Machine Learning Risk Scoring:** Replace rule-based AI with a trained machine learning model that learns access patterns over time for more nuanced and adaptive risk classification.
- **Solar Power Backup:** Add solar panel and battery backup to ensure the system continues operating during power outages, critical for bank vault security continuity.

## 6.2 Problem Solution

| Problem encountered                                                                 | Root Cause                                                                                                         | Solution Applied                                                                                                                                                                |
|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Arduino uno failed to upload (programmer not responding and couldn't find the port) | Arduino Uno bootloader corrupted or ATmega16U2 USB chip faulty. Access denied error on COM port.                   | Switched to ESP32-only architecture. All components rewired to ESP32 GPIO pins, eliminating the need for Arduino Uno entirely.                                                  |
| MQ-2 gas sensor triggering falsely on power on                                      | The MQ-2 heating element is cold on startup and gives unstable readings for the first 30 seconds.                  | Make sure the MQ-2 gas sensor had a 30 seconds warmup period with countdown on LCD. Implemented 15 seconds cooldown timer to prevent repeated alerts from continuous detection. |
| PIR sensor stuck HIGH (always detecting motion)                                     | Sensitivity potentiometer set too high. The sensor was picking up ambient heat and sunlight.                       | Adjusted left potentiometer anticlockwise to minimum sensitivity. Set jumper to L (single trigger) mode. Added 10 second motion cooldown timer.                                 |
| Flask dashboard does not update following real time                                 | ESP32 was POSTing to the wrong IP address(hotspot IP changed after reconnect) Windows firewall blocking port 5000. | Added Windows Firewall inbound rule for TCP port 5000 using netsh command. Updated flaskURL in sketch with correct IPv4 address after each hotspot reconnects.                  |
| Risk count does not increase and keep showing 0 on dashboard                        | Jinja2 selectattr filter using wrong tuple index for risk_score column.                                            | Moved risk counting logic to Python in <a href="#">app.py</a> using sum() comprehension (log[-1]=='LOW') and passed counts directly to template as variables.                   |
| Keypad middle columns stop responding (keys 2,5,8,0)                                | Pin 4 on Arduino was faulty. Ribbon cable may have been misaligned by one slot.                                    | Moved keypad Column 2 wire to GPIO 25 on ESP32. Re-mapped colPins array. Verified all keys functional after rewiring.                                                           |
| MEDIUM risk Telegram alert delayed or missing                                       | send Telegram() was called after openDoor() which blocks for 5 seconds,                                            | Restructured MEDIUM risk code block to call sendTelegram() and                                                                                                                  |

|  |                                          |                                                                              |
|--|------------------------------------------|------------------------------------------------------------------------------|
|  | causing the alert to be delayed or lost. | sendToFlask() before calling openDoor(). Alert now fires before servo moves. |
|--|------------------------------------------|------------------------------------------------------------------------------|

## 7.0 REFERENCES

1. Risco, "Seismic & Vibration Detectors for Vault & ATM Security | RISCO," Nov. 2025. Accessed: July 25, 2026. [Online]. Available: <https://riscogroup.com/en/redefining-vault-security/>
2. "ESP32 Useful Wi-Fi Library Functions (Arduino IDE) | Random Nerd Tutorials," Feb. 2021. Accessed: July 25, 2026. [Online]. Available: <https://randomnerdtutorials.com/esp32-useful-wi-fi-functions-arduino/>
3. Mausam678, "GitHub - Mausam678/ESP32-WiFi-Connection-Example: This repository contains a simple example demonstrating how to connect an ESP32 microcontroller to a Wi-Fi network using the Arduino IDE. The code connects to a Wi-Fi network, monitors the connection progress, and retrieves the ESP32's IP address along with the router's gateway IP.," *GitHub*, 2026. <https://github.com/mausam678/esp32-wifi-connection-example> (accessed July 25, 2026).
4. B. Andrei, "Building a Real-Time Client Monitoring System with Flask and Socket.IO," *Medium*, May 31, 2024. <https://medium.com/@boata.andrei88/building-a-real-time-client-monitoring-system-with-flask-and-socket-io-c023fba5e26c> (accessed July 25, 2026).
5. ASECstaff, "Modular Vaults vs Traditional Vaults," *Associated Security*, Apr. 28, 2025. <https://www.associatedsecurity.co.uk/news/modular-vaults-vs-traditional-vaults/> (accessed July 25, 2026).